

CS & IT ENGINEERING



Computer Network - 2

Framing & Error Control

Lecture No. - 03

By - Abhishek Sir





Recap of Previous Lecture



Topic

Bit Stuffing ✓

Topic

Two-dimension (2D) Parity





Topics to be Covered



Topic

Two-dimension (2D) Parity

Topic

Hamming Code



[MCQ]

[GATE-2014][1 Mark]



#Q. A bit-stuffing based framing protocol uses an 8-bit delimiter pattern of '01111110'. If the output bit-string after stuffing is 01111100101, then the input bit-string is :

☒ A 0111110100

☒ B 0111110101

☒ C 0111111101

☒ D 0111111111

Ans: B

Frame delimiter

=

01111110

Output

=

01111100101

Transmitted data

=

01111110 01111100101 01111110

↑
SFD (Flag)

↓
discard

↓
EFD (Flag)

Input (Frame)

=

01111100101

~~01111110~~

↓
Ans

(Recv
AT)

[MCQ]

[GATE-IT-2004] [1 Mark]



#Q. In a data link protocol, the frame delimiter flag is given by 0111. Assuming that bit stuffing is employed, the transmitter sends the data sequence 01110110 as:

- ☒ A 01101011
- ☒ B 011010110
- ☒ C 011101100
- ☒ D 0110101100

Ans: D

Frame delimiter = 0111 / 01110

Input (Frame) = 01110110

Transmitted data = 01110 0110 101100 01110
 SFD EFD

Output = 0110101100

Output = Input with bit stuffing



Topic : Two-dimension (2D) Parity

→ Both **sender** and **receiver** must agree on **same block size** and **same parity** (either **even** or **odd** parity)

→ **Block Size** = $m * n$
[**m rows** & **n columns**]

$$\text{Number of Data bits} = (m-1) * (n-1)$$

$$\text{Number of parity bits} = [m + n - 1]$$

[No. of parity bits
is depend on
no of data bits]

⇒ Single bit Error correction



Topic : Two-dimension (2D) Parity



$$R_1 = d_1 \oplus d_2 \oplus d_3 \oplus d_4 \oplus d_5$$

$$R_2 = d_6 \oplus d_7 \oplus d_8 \oplus d_9 \oplus d_{10}$$

$$R_3 = d_{11} \oplus d_{12} \oplus d_{13} \oplus d_{14} \oplus d_{15}$$

$$C_1 = d_1 \oplus d_6 \oplus d_{11}$$

$$C_2 = d_2 \oplus d_7 \oplus d_{12}$$

$$C_3 = d_3 \oplus d_8 \oplus d_{13}$$

$$C_4 = d_4 \oplus d_9 \oplus d_{14}$$

$$C_5 = d_5 \oplus d_{10} \oplus d_{15}$$

$$P = R_1 \oplus R_2 \oplus R_3 = C_1 \oplus C_2 \oplus C_3 \oplus C_4 \oplus C_5$$

Suppose Block Size = 4 x 6 and "Even Parity"

Data = "d₁d₂d₃d₄d₅ d₆d₇d₈d₉d₁₀ d₁₁d₁₂d₁₃d₁₄d₁₅"

Sender

d ₁	d ₂	d ₃	d ₄	d ₅	R ₁
d ₆	d ₇	d ₈	d ₉	d ₁₀	R ₂
d ₁₁	d ₁₂	d ₁₃	d ₁₄	d ₁₅	R ₃
C ₁	C ₂	C ₃	C ₄	C ₅	P

Transmitted Data = "d₁d₂d₃d₄d₅R₁ d₆d₇d₈d₉d₁₀R₂ d₁₁d₁₂d₁₃d₁₄d₁₅R₃ C₁C₂C₃C₄C₅P"



Topic : Two-dimension (2D) Parity



CASE I : No any error

Suppose Block Size = 4 x 6 and "Even Parity"

Receiver

1	0	0	1	1	1	✓
0	1	1	0	1	1	✓
1	0	0	1	0	0	✓
0	1	1	0	0	0	✓
✓	✓	✓	✓	✓	✓	

Transmitted Data = "1 0 0 1 1 1 0 1 1 0 1 1 1 0 0 1 0 0 0 1 1 0 0 0"

Received Data = "1 0 0 1 1 1 0 1 1 0 1 1 1 0 0 1 0 0 0 1 1 0 0 0"

Receiver Concluded : No any Error detected; Accept the data;



Topic : Two-dimension (2D) Parity



Error Detection:-

if receiver finds all row wise and column wise parities are balanced
then receiver concluded "No any error detected"

else

receiver concluded "Error detected", call Error correction;



Topic : Two-dimension (2D) Parity



CASE I : No any error





Topic : Two-dimension (2D) Parity



CASE II : One-bit error

Suppose Block Size = 4 x 6 and "Even Parity"

Receiver

1	0	0	1	1	1	✓
0	1	0	0	1	1	←
1	0	0	1	0	0	✓
0	1	1	0	0	0	✓
✓	✓	↑	✓	✓	✓	

Transmitted Data = "1 0 0 1 1 1 0 1 1 0 1 1 1 0 0 1 0 0 0 0 1 1 0 0 0 0"

Received Data = "1 0 0 1 1 1 0 1 0 0 1 1 1 0 0 1 0 0 0 0 0 1 1 0 0 0 0"

Receiver Concluded : Error Detected; Error Correction: Single bit Error case
⇒ Error corrected successfully

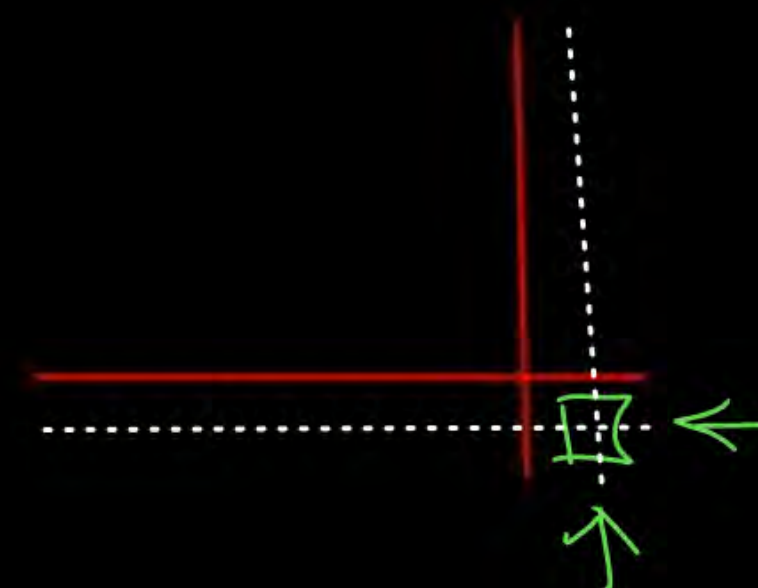
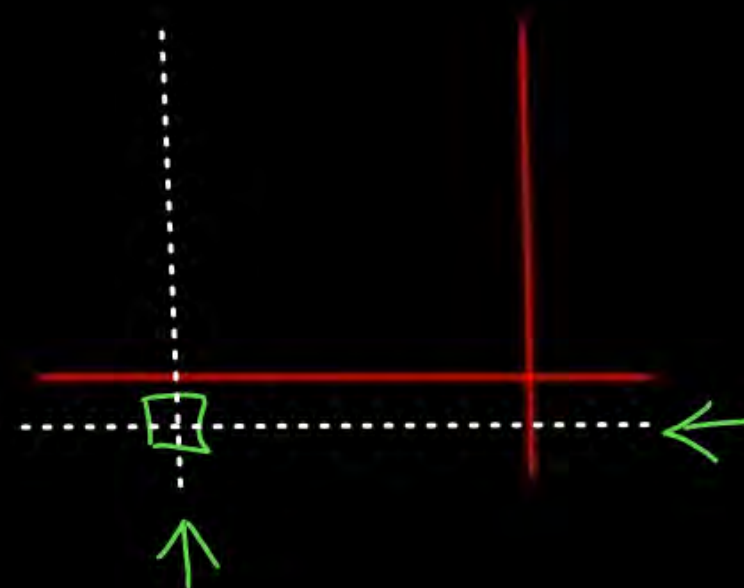
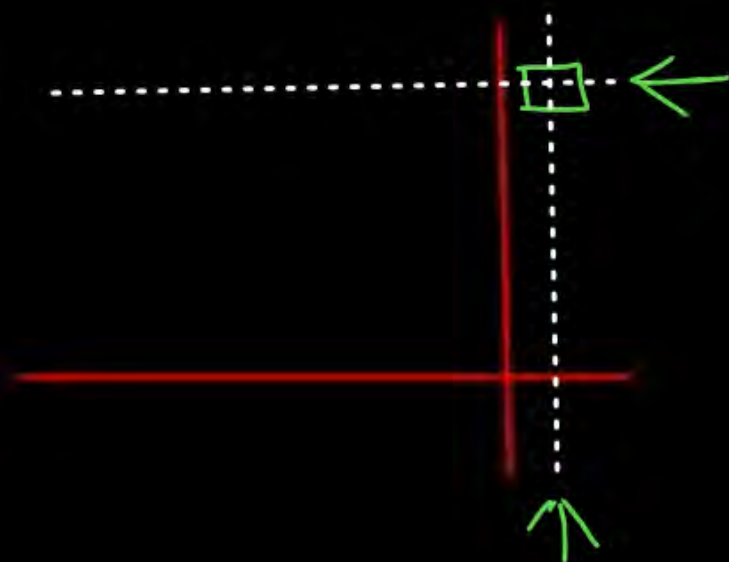


Topic : Two-dimension (2D) Parity



CASE II : One-bit error

⇒ Always detected and corrected successfully





Topic : Two-dimension (2D) Parity



CASE III : Two-bit error

Suppose Block Size = 4 x 6 and "Even Parity"

Receiver

1	0	0	1	1	1	✓
0	1	0	0	1	1	←
1	1	0	1	0	0	←
0	1	1	0	0	0	✓
✓	↑	↑	✓	✓	✓	

Transmitted Data = "1 0 0 1 1 1 0 1 1 0 1 1 1 0 0 1 0 0 0 1 1 0 0 0"

Received Data = "1 0 0 1 1 1 0 1 0 0 1 1 1 1 0 1 0 0 0 1 1 0 0 0"

Receiver Concluded : Error Detected; Error Correction: Min^m two bit Error
Unable to correct;

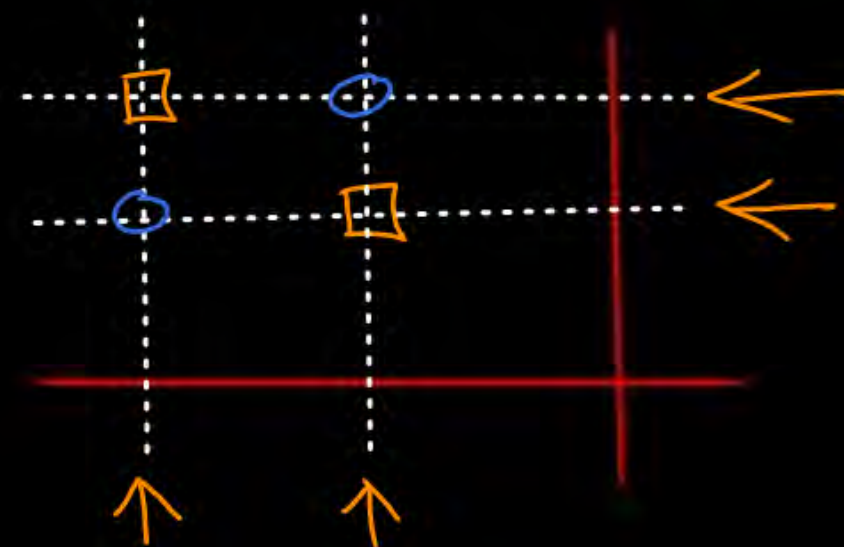
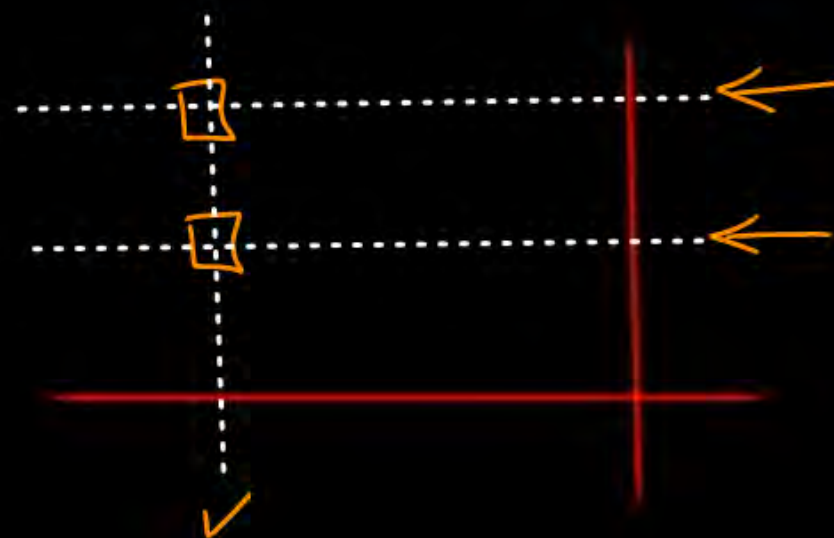
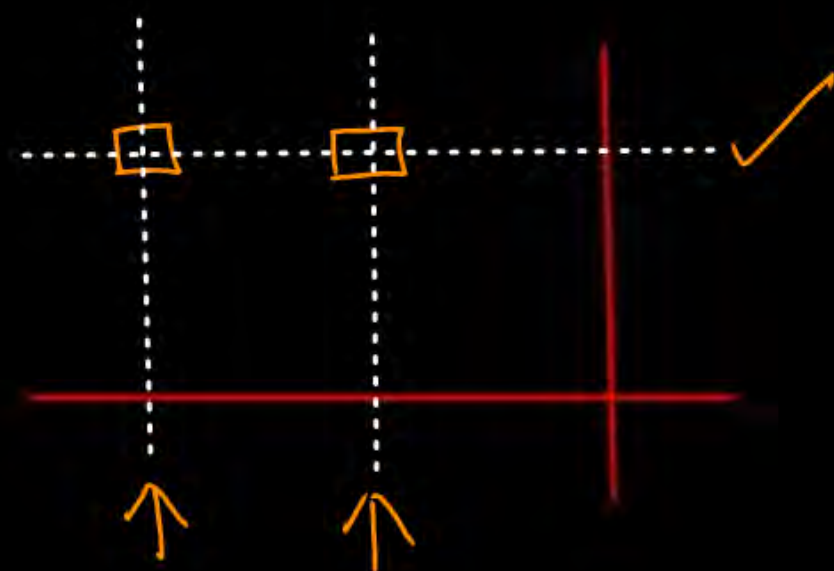


Topic : Two-dimension (2D) Parity



CASE III : Two-bit error

→ Always detected, Unable to correct



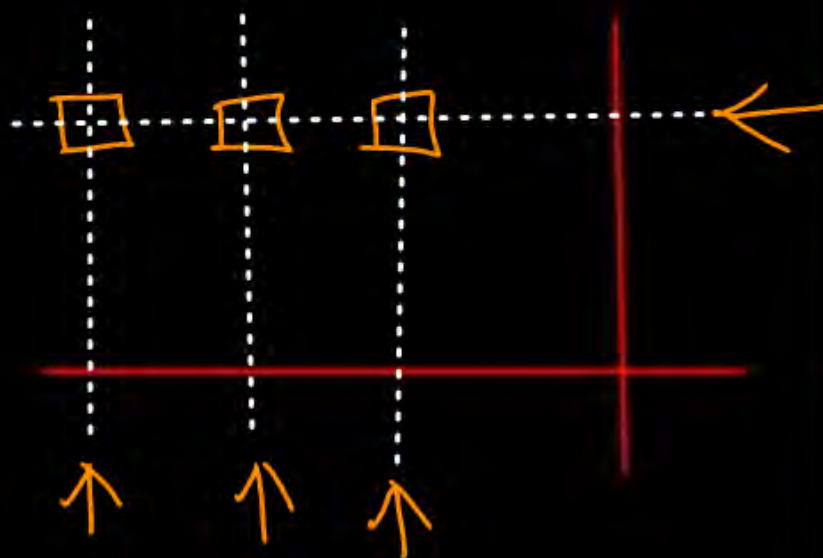


Topic : Two-dimension (2D) Parity

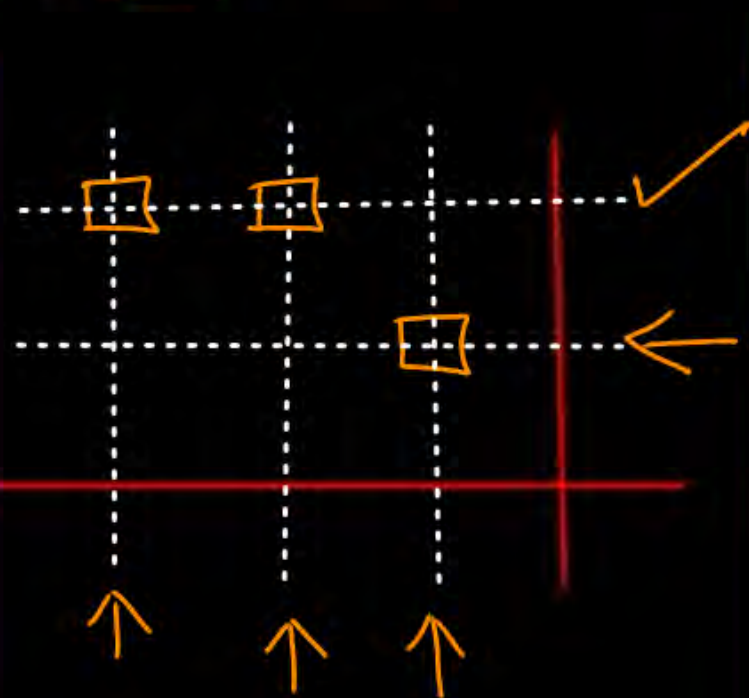


CASE IV : Three-bit error

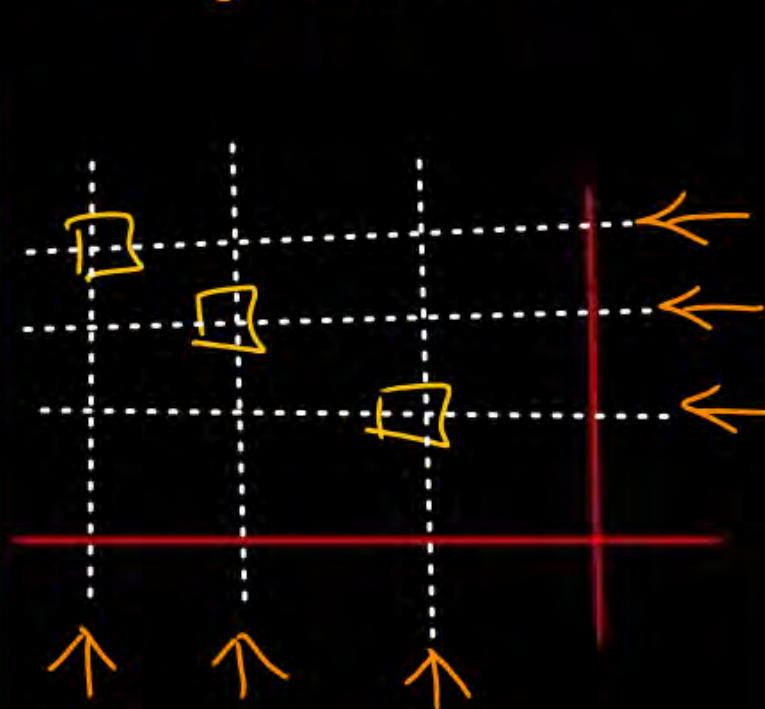
Always detected,
Unable to correct,



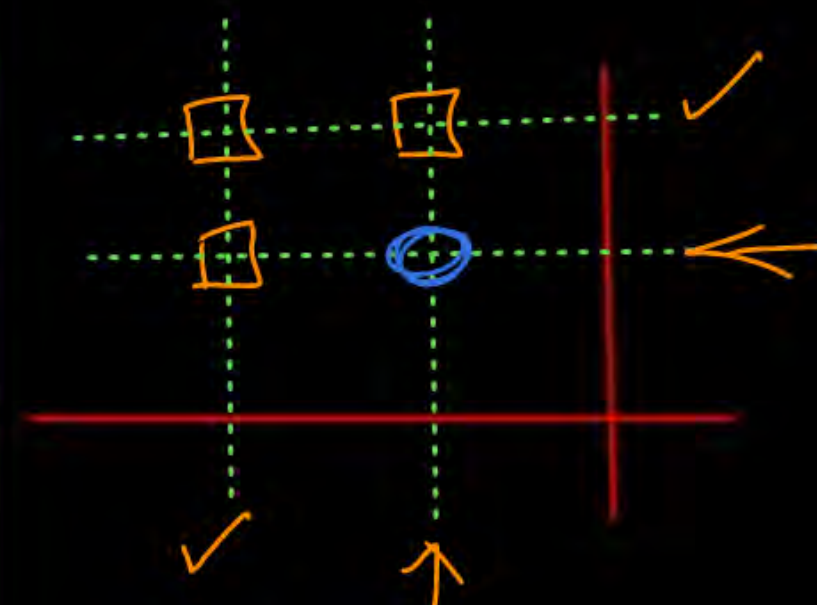
CASE-A



CASE-B



CASE-C



CASE-D
single bit Error
Corrected
Successfully

CASE A, B & C $\Rightarrow$ Min^m three bit error
(Unable to correct)

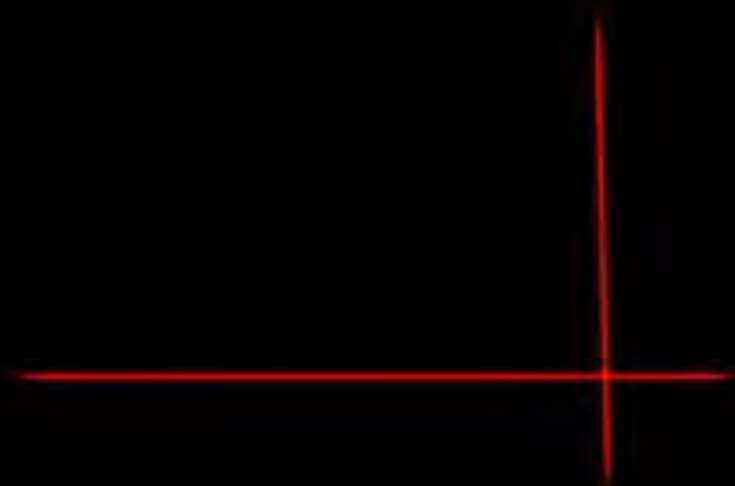
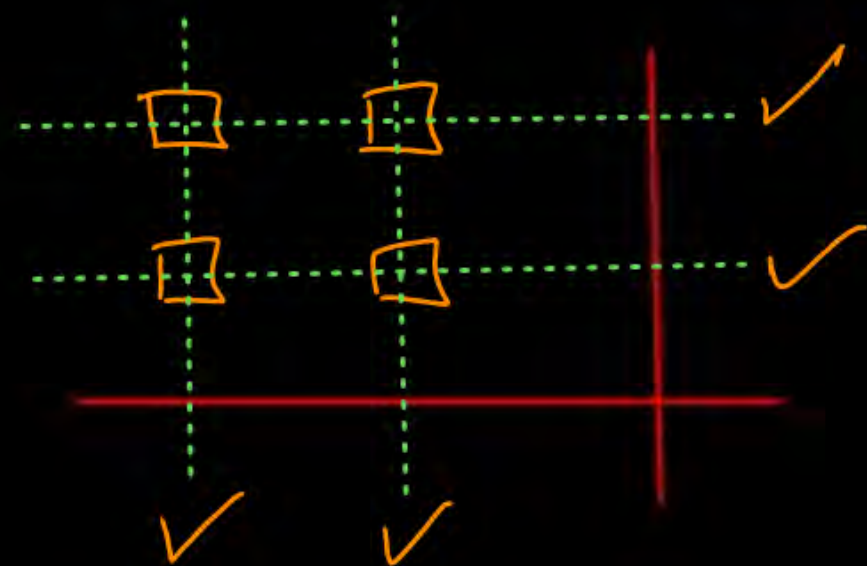


Topic : Two-dimension (2D) Parity



CASE V : Four-bit error

→ May be detected;



Example :-

#Q. Consider 2D parity scheme for error detection using the (Even parity), the block size is 4 x 6 (including parity bits) contains (15 data bits). The matrix shows data received by a receiver and has n corrupted bits. What is the minimum possible value of n?

(Even Parity)

1	1	0	1	1	1	←
0	1	1	0	1	1	✓
1	0	0	0	0	0	←
0	1	1	0	0	0	✓
✓	↑	✓	↑	✓	✓	



Topic : Two-dimension (2D) Parity



Minimum number of bits corrupted in the Block

= Maximum [total number of row wise parity unbalanced,
total number of column wise parity unbalanced]

$$= \text{Max}[2, 2]$$

$$= 2$$

Ans = 2

[MCQ]

IISC, H.W.

[GATE-IT-2008] [2 Mark]

#Q. Data transmitted on a link uses the following 2D parity scheme for error detection :

Each sequence of 28 bits is arranged in a 4 x 7 matrix (rows r_0 through r_3 , and columns d_7 through d_1) and is padded with a column d_0 and row r_4 of parity bits computed using the Even parity scheme. Each bit of column d_0 (respectively, row r_4) gives the parity of the corresponding row (respectively, column). These 40 bits are transmitted over the data link.

The table shows data received by a receiver and has n corrupted bits. What is the minimum possible value of n ?

- A** 1
- B** 2
- C** 3
- D** 4

	d_7	d_6	d_5	d_4	d_3	d_2	d_1	d_0
r_0	0	1	0	1	0	0	1	1
r_1	1	1	0	0	1	1	1	0
r_2	0	0	0	1	0	1	0	0
r_3	0	1	1	0	1	0	1	0
r_4	1	1	0	0	0	1	1	0



Topic : Two-dimension (2D) Parity



→ Receiver detect and correct "all single bit error"

→ In case of burst error,
receiver may able to detect "burst error"

$d_1 d_2$	R_1
$d_3 d_4$	R_2
$c_1 c_2$	P



Topic : Two-dimension (2D) Parity

* Linear code



Two-dimension Parity (with Even Parity) and 4 data bits:

Data	→	Codeword	Data	→	Codeword
$d_1d_2d_3d_4$	→	$d_1d_2R_1d_3d_4R_2C_1C_2P$	$d_1d_2d_3d_4$	→	$d_1d_2R_1d_3d_4R_2C_1C_2P$
0 0 0 0	→	0 0 0 0 0 0 0 0	1 0 0 0	→	1 0 1 0 0 0 1 0
0 0 0 1	→	0 0 0 0 1 1 0 1	1 0 0 1	→	1 0 1 0 1 1 1 0
0 0 1 0	→	0 0 0 1 0 1 1 0	1 0 1 0	→	1 0 1 1 0 1 0 0
0 0 1 1	→	0 0 0 1 1 0 1 1	1 0 1 1	→	1 0 1 1 1 0 0 1
0 1 0 0	→	0 1 1 0 0 0 0 1	1 1 0 0	→	1 1 0 0 0 0 1 1
0 1 0 1	→	0 1 1 0 1 1 0 0	1 1 0 1	→	1 1 0 0 1 1 1 0
0 1 1 0	→	0 1 1 1 0 1 1 1	1 1 1 0	→	1 1 0 1 0 1 0 1
0 1 1 1	→	0 1 1 1 1 0 1 0	1 1 1 1	→	1 1 0 1 1 0 0 0

odd parity (4x5)

0	1	0	1	1
1	0	1	1	0
0	1	1	0	1
0	1	1	1	0 1

Even parity

0	1	0	1	0
1	0	1	1	1
0	1	1	0	0
1	0	0	0	1



Topic : Hamming Code



- Single bit error-correcting code
- Both transmitter and receiver must agree on same parity
[either "Even Parity" or "Odd Parity"]



Topic : Hamming Code



→ Parity bit (R_i) placed at position
[where $i = 0, 1, 2, 3 \dots$]

=

2^i

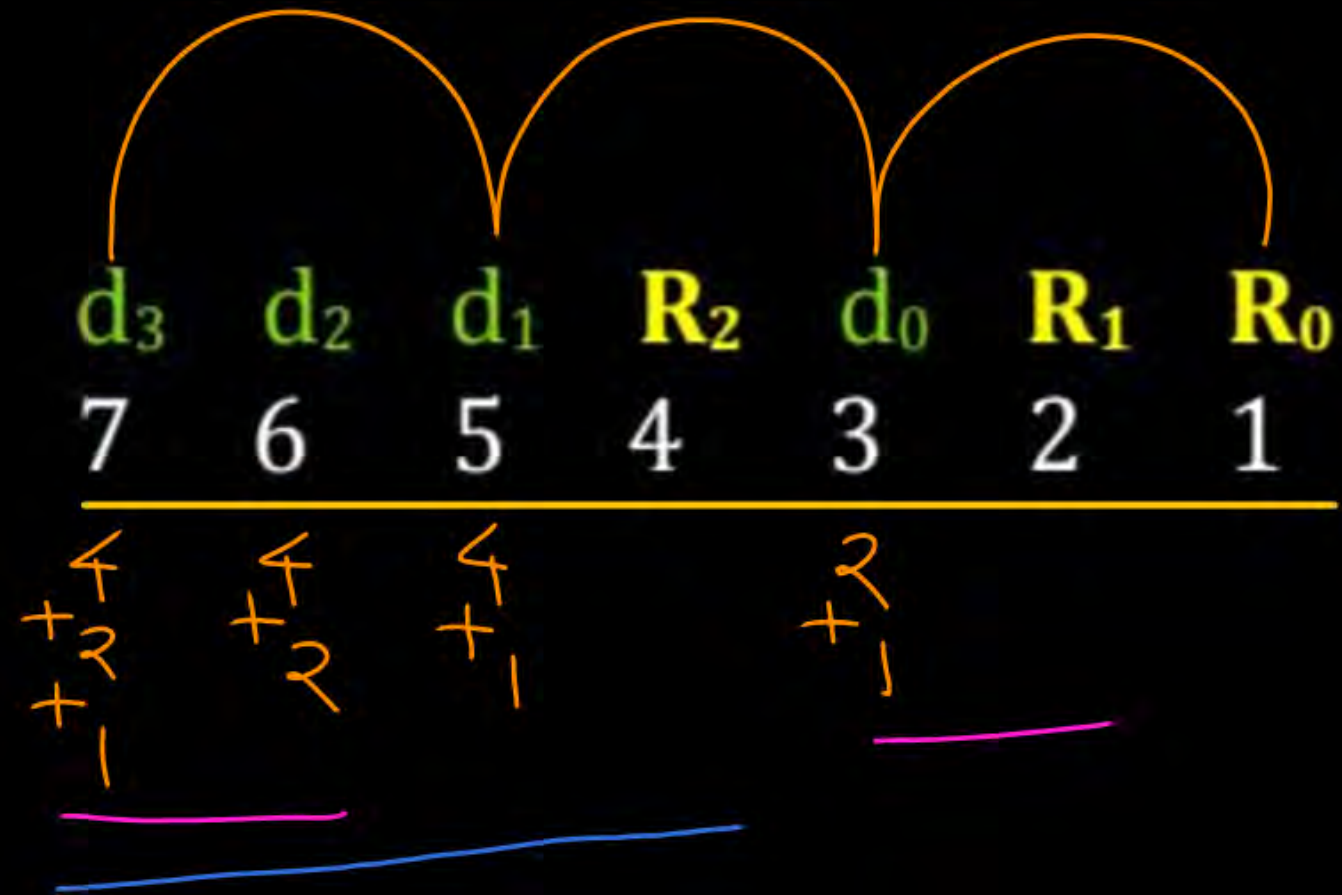
Data $\Rightarrow$
 $d_3 d_2 d_1 d_0$

d_3	d_2	d_1	R_2	d_0	R_1	R_0
7	6	5	<u>4</u>	3	<u>2</u>	<u>1</u>
			2^2		2^1	2^0

→ First parity bit placed at least significant bit position.
 R_0



Topic : Hamming Code

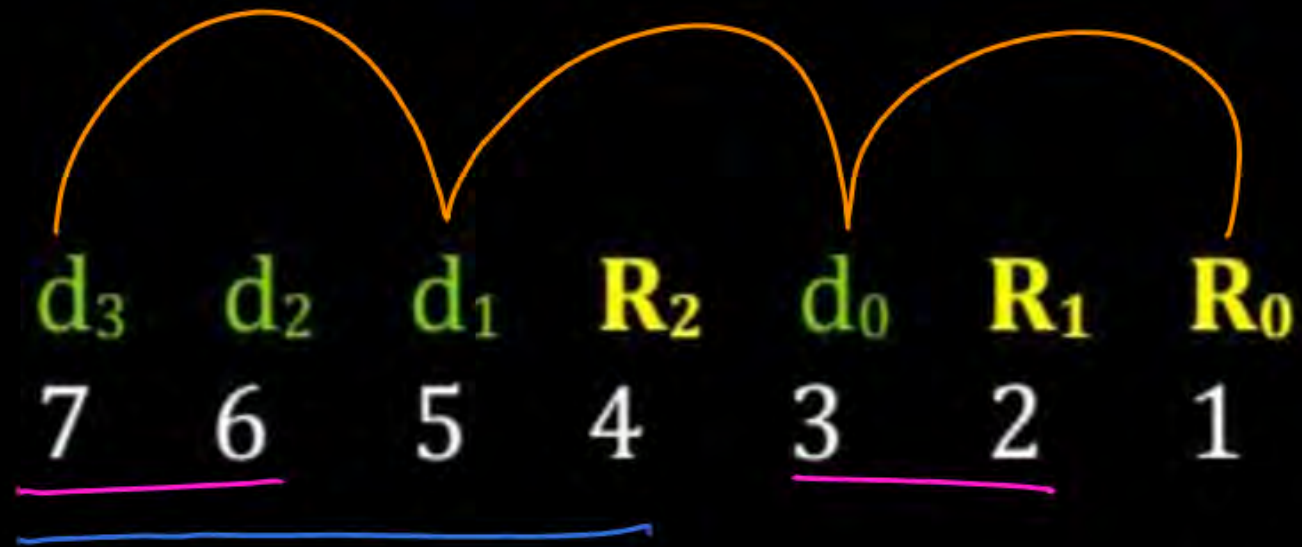




Topic : Hamming Code



→ Even Parity



$$R_0 = d_0 \oplus d_1 \oplus d_3$$

$$[1 = 3, 5, 7]$$

$$R_1 = d_0 \oplus d_2 \oplus d_3$$

$$[2 = 3, 6, 7]$$

$$R_2 = d_1 \oplus d_2 \oplus d_3$$

$$[4 = 5, 6, 7]$$

Example 1 :-

#Q. Consider hamming code error control technique which uses 'even parity' and first parity bit placed at least significant bit position. The four bits data '0101' (placed from most significant bit to least significant bit position) encoded as :

Solution 1 :-

(Even Parity)

Data $\rightarrow$ 0101



$$R_0 = 1$$
$$R_1 = 0$$
$$R_2 = 1$$

Solution 1 :- (Even Parity)

<u>0</u>	<u>1</u>	<u>0</u>	<u>1</u>	<u>1</u>	<u>0</u>	<u>1</u>
7	6	5	4	3	2	1

Ans : 0101101

Hamming Code
Transmitted Code

Example 2 :-

#Q. Consider hamming code error control technique which uses 'even parity' and first parity bit placed at least significant bit position. The four bits data '1010' (placed from most significant bit to least significant bit position) encoded as :

Solution 2 :-

(Even Parity)

Data $\rightarrow$ 1010



$$\begin{aligned} P_0 &= 0 \\ P_1 &= 1 \\ P_2 &= 0 \end{aligned}$$

Solution 2 :- (Even Parity)

<u>1</u>	<u>0</u>	<u>1</u>	<u>0</u>	<u>0</u>	<u>1</u>	<u>0</u>
7	6	5	4	3	2	1

Ans : 1010010



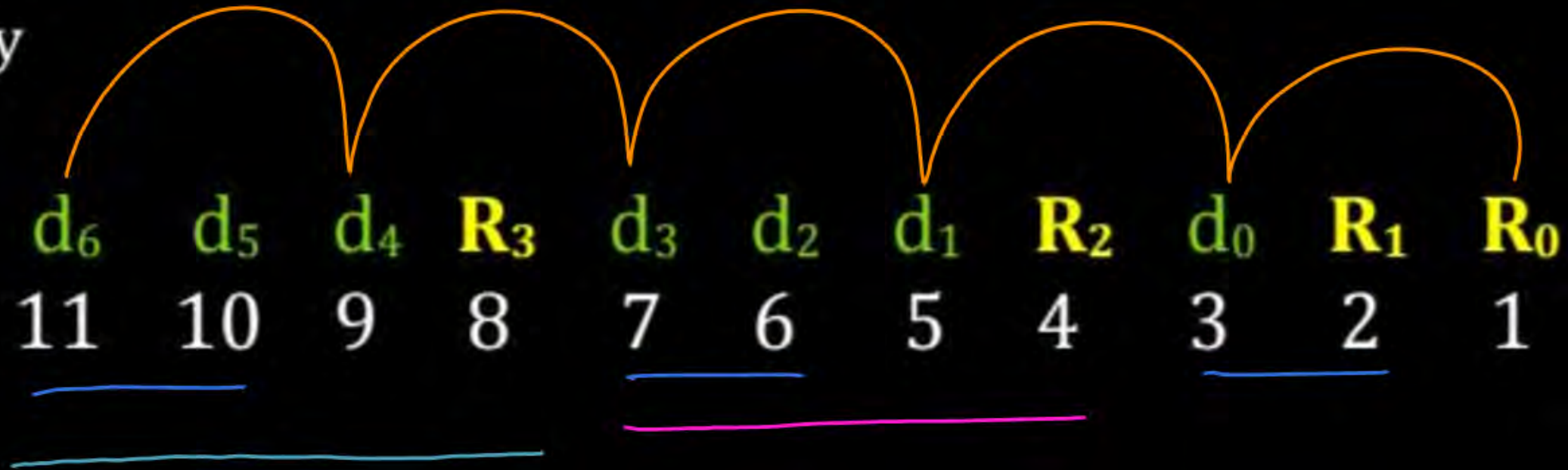
→ First parity bit placed at least significant bit position.



Topic : Hamming Code



→ Even Parity



$$R_0 = d_0 \oplus d_1 \oplus d_3 \oplus d_4 \oplus d_6$$

$$R_1 = d_0 \oplus d_2 \oplus d_3 \oplus d_5 \oplus d_6$$

$$R_2 = d_1 \oplus d_2 \oplus d_3$$

$$R_3 = d_4 \oplus d_5 \oplus d_6$$

$$[1 = 3, 5, 7, 9, 11]$$

$$[2 = 3, 6, 7, 10, 11]$$

$$[4 = 5, 6, 7]$$

$$[8 = 9, 10, 11]$$

Example 3 :- (H.W.)

#Q. Consider hamming code error control technique which uses 'even parity' and first parity bit placed at least significant bit position. The seven bits data '1101101' (placed from most significant bit to least significant bit position) encoded as :



2 mins Summary



Topic

Two-dimension (2D) Parity ✓

Topic

Hamming Code →



THANK - YOU

